



AI Engineering Techniques & Architectures Project: **Meridian Strategy Group** **People & Travel Operations Assistant**

Software Requirements Specification

Dineo Malatjie

+27 72 218 2350 | dineo.malatjie@gmail.com

AI Disclosure

- Generative AI tools were used during this project as part of the engineering workflow. OpenAI ChatGPT assisted with planning, drafting and review during the project's development.
- The author remains responsible for the project concept, requirements, architecture, implementation decisions, validation, evaluation and submitted content.
- AI-generated outputs informed the design and development process but did not determine the final project decisions.
- Each recommendation was assessed and revised using the project requirements, implementation evidence and observed system behaviour.
- ChatGPT assisted with defining and refining the project concept, use case, business requirements and software requirements. It also helped compare architecture and design options, identify edge cases and evaluation considerations, improve requirements traceability, explain technical concepts such as retrieval-augmented generation (RAG), agent orchestration, Model Context Protocol (MCP), deployment, CI/CD and evaluation, and review project artefacts and documentation.
- AI coding tools assisted with code generation, debugging, testing and repository-level verification. All generated code and technical recommendations were reviewed, tested and adjusted before use.
- The accompanying AI Tooling documentation identifies the tools used, their roles and the relevant AI interactions.

System Overview

Meridian Compass is a web-based agentic employee-support application combining Enterprise Identity, Retrieval-Augmented Generation, structured operational data and tool-enabled agent orchestration.

Functional Requirements

ID	Functional requirement
FR-01	Synthetic Enterprise Identity selector/login and server-side session context.
FR-02	Employee-facing chat UI.
FR-03	POST /chat returns the canonical response fields.
FR-04	GET /health reports application, MCP and RAG readiness.
FR-05	Agent interprets intent and selects RAG, MCP tools, clarification or safe insufficiency handling.
FR-06	Approved canonical Markdown is authoritative RAG input; duplicate representations are not double-indexed.
FR-07	Preserve document metadata where available.
FR-08	Retrieval top-k is configurable with baseline 5.
FR-09	Discover/invoke MCP tools through the configured MCP boundary.
FR-10	Orchestrator must not directly read MCP-owned operational JSON.
FR-11	Material MSG policy/procedure claims undergo grounding verification.
FR-12	Privacy uses authenticated identity; prompt text cannot override identity.
FR-13	Establish supported approval role before resolving a named person.
FR-14	Missing evidence produces explicit insufficiency rather than inference.
FR-15	Consequential mock actions require confirmation and transient/resettable state.
FR-16	No internet/external policy/general model knowledge may establish Meridian policy.
FR-17	Logs/traces expose architectural events, not hidden chain-of-thought.

Canonical API Contract

- POST /chat request: {"message":"string","session_id":"string","confirm_action":false}. Authenticated identity is resolved server-side.
- Response fields are exactly: answer, status, citations, source_snippets and tool_trace.

Public Statuses

Status	Use
answered	Supported evidence is sufficient.

insufficient_evidence	Company evidence is not sufficient.
clarification_required	User detail/correction is required.
action_confirmation_required	A consequential mock action needs explicit confirmation.
forbidden	Authorisation/privacy rule prevents the request.
not_found	Required authorised data is not found.
out_of_scope	Request is unrelated to supported Meridian work.
tool_error	A required dependency/tool failed.

Health Contract

Healthy target:

```
{"status":"healthy","application":"meridiancompass","mcp":"connected","rag":"ready"}.
```

RAG Requirements and Realised Design

Requirement	Final implementation
Authoritative corpus	19 canonical Markdown policy/procedure documents
Metadata	Document, title, type/topic, section and effective-date metadata where available
Retrieval	Configurable top-k, baseline k=5
Embeddings	sentence-transformers/all-MiniLM-L6-v2, 384 dimensions
Vector store	FAISS IndexFlatIP
Index	353 chunks; built during Docker image construction
Failure behaviour	Missing/stale/corrupt/incompatible index fails closed

MCP Requirements and Realised Design

The MCP layer uses the official MCP Python SDK and Streamable HTTP. In production, the logical MCP protocol boundary is mounted in-process at /mcp; the application uses the official ClientSession/Streamable HTTP protocol rather than a localhost TCP service. Tool discovery is implemented and tested separately; it does not run on every /chat request.

The final MCP surface contains 13 tools: one knowledge search tool, nine structured read tools and three controlled mock action tools.

Security, Privacy and Grounding

Authenticated identity is server-authoritative and cannot be overridden by prompt text. Ordinary employees cannot retrieve another employee's private HR data merely by naming them.

Policy/procedure claims require retrieved approved evidence.
Operational claims require matching authorised structured facts.
Invalid citations, unsupported claims, provider failures and insufficient evidence fail closed.

Portability and CI/CD

The solution runs locally, in Docker, in GitHub Actions and on Render. CI on Python 3.12 installs dependencies, builds the canonical RAG index, runs the automated suite, verifies policy retrieval, performs real MCP discovery and a safe call, builds/starts Docker and verifies /health.

Verification Status

Final automated inventory: 138 tests collected with no skip/xfail markers in the final audit.
Controlled evaluation: 27/30 passed (90%). Production acceptance completed the PTO workflow with live LLM synthesis, MCP execution, RAG evidence, structured context, citation display and system trace.